



# HITMAN SYNDICATE NETWORK

## PROJECT REPORT

Name: Safwan Salehin  
Roll: 2207044  
Course NO: CSE-3100  
Course Title: Web Programming Laboratory  
Group: A2  
Date: July 19<sup>th</sup>, 2026

### **SUBMITTED TO:**

**Kazi Saeed Alam**

Assistant Professor

Computer Science and Engineering  
Khulna University of Engineering & Technology

**Ehsanul Karim**

Lecturer

Computer Science and Engineering  
Khulna University of Engineering & Technology

## Summary

The Hitman Syndicate Network Terminal is a state-of-the-art secure web portal engineered to coordinate global operations for the Syndicate's elite operatives. Built upon the robust Laravel 12 framework, the portal features comprehensive session-based security, cookie encryption, role-based security clearance groups, automated operative onboarding, dynamic meteorological and geographical location telemetry, operational contract distributions, and financial audit logging.

This report provides an in-depth overview of the systems, database schemas, feature enhancements, and test verifications implemented during the current development lifecycle.

## System Architecture

The application architecture follows modern MVC design principles, separating business logic, data models, and presentation layers while maintaining premium performance and micro-second responsiveness.

## Technology Stack

- **Core Web Framework:** Laravel 12 (Breeze Starter Kit for authentication scaffolding)
- **Programming Languages:** PHP 8.2+ (Backend logic & routing), JavaScript ES6 (Client-side interactive modals & geolocation telemetry)
- **Styles & Theming:** Tailwind CSS (Vite asset pipeline) featuring a custom Syndicate Network dark terminal user interface theme
- **Database Engine:** MySQL (Production environment), SQLite in-memory (Test suite automation)
- **Testing Suite:** Pest PHP Framework for automated unit and feature verification

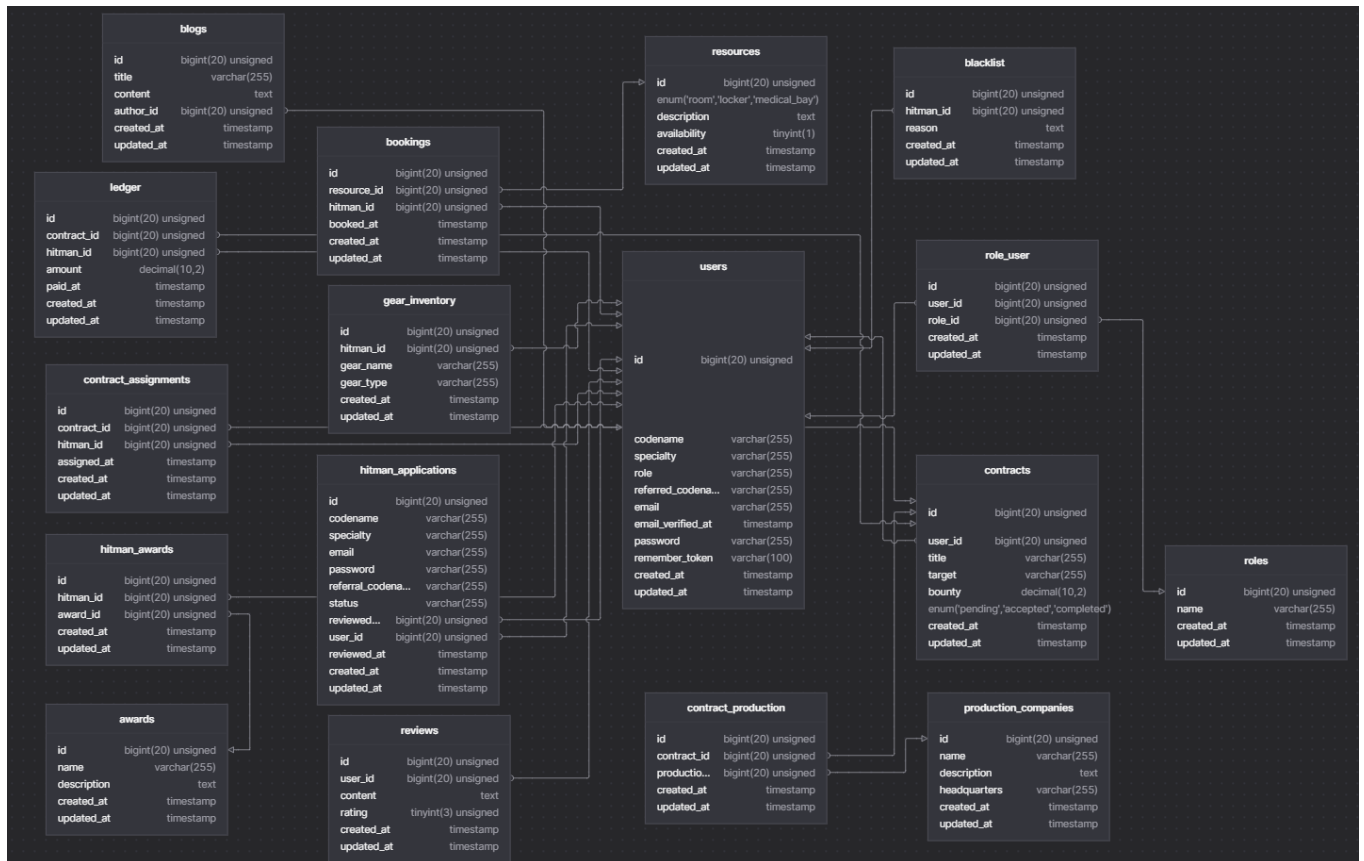
## Key Operational Flows

- **Onboarding & Vetting Flow:** New operatives register on the portal, creating a pending application. Administrators review applicant profiles and codenames, verifying security clearance before granting active 'Hitman' clearance status.
- **Contract Claiming & Acceptance Flow:** Admin deploys new contracts into the operations pool as unassigned (`user_id = null`) and pending. Hitman operatives claim contracts directly from their dashboard. Upon acceptance, the operative's foreign key (`user_id`) is set on the contract, and the status shifts to 'accepted'.
- **Contract Completion & Ledger Audit:** An operative marks their accepted contract as completed. This automatically locks the contract, updates its database status to 'completed', and initiates a ledger transaction recording the target contract, claiming operative, bounty amount, and payout timestamp.

## Database Details

The relational database schema enforces strict referential integrity and cascading behaviors to guarantee system state consistency.

Schema Diagram:



## Key Implementation Details

### Secure Session Lifecycle Management

Fully audited Breeze auth middleware and configured cookie parameters. Active features for session cookie lifetimes, encryption layers, session fixation prevention (via session regeneration upon login), and session database purging during account deletion.

Technical Specifics:

- Session driver configured to 'database' in config/session.php, enabling the system to inspect and invalidate sessions on multi-device scenarios.
- Session regeneration triggers immediately upon successful authentication via `$request->session()->regenerate()` inside the `AuthenticatedSessionController.php`, defending the system against session fixation exploits.
- Strict invalidation methods are invoked at account termination and logouts via `$request->session()->invalidate()` and `$request->session()->regenerateToken()` to clear session ciphers and prevent CSRF replay vectors.
- Cookie configurations enforce strict security parameters: `HttpOnly` is set to `true` (preventing

access by cross-site scripting), SameSite is configured as 'Lax' to restrict cross-site sharing, and Secure attributes force HTTPS encryption transit.

The image displays two screenshots of a web application interface for 'THE HITMAN'S SYNDICATE'.

The left screenshot shows the 'OPERATIVE AUTHORIZATION' page. It features a logo at the top, followed by the title 'OPERATIVE AUTHORIZATION'. Below this is a form with fields for 'SECURE EMAIL' (containing 'agency@secure.net'), 'ACCESS KEY', and a 'FORGOT KEY?' link. There is a checkbox for 'MAINTAIN SESSION' and an orange 'ACCESS' button. At the bottom, a link reads 'NEW OPERATIVE? REQUEST MEMBERSHIP'.

The right screenshot shows the 'REQUEST MEMBERSHIP' page. It features the same logo and title 'REQUEST MEMBERSHIP'. Below the title is a subtitle: 'Every operative starts with an application. Submit your credentials and secure referral to initiate onboarding.' The form is divided into sections: 'OPERATIVE PROFILE' with fields for 'CODENAME' (e.g., Agent 47) and 'SPECIALTY / SKILL' (e.g., Infiltration); 'SECURE EMAIL' (agency@secure.net) and 'REFERRAL CODENAME' (Active Member Name); 'SECURITY PHASE'; 'ACCESS PASSWORD' and 'CONFIRM PASSWORD'; and a 'SUBMIT APPLICATION' button. At the bottom, a link reads 'ALREADY APPROVED? LOG IN'.

## Dynamic Location Telemetry Widget

Developed a geolocation client script integrating the OpenWeatherMap API key to identify the operative's location and overlay it in the header. Added an IP-based location fallback via 'ipapi.co' when browser permissions are withheld, adding a proxy indicator label.

Technical Specifics:

- Frontend scripts utilize the HTML5 Geolocation API (`navigator.geolocation`) to query physical coordinates (latitude and longitude) of the active terminal.
- Upon resolving coordinates, the client executes an asynchronous fetch request querying the OpenWeatherMap API endpoint: `https://api.openweathermap.org/data/2.5/weather`, using a private key.
- The widget extracts city name, country code, and temperature metric (Celsius) from the returned JSON response, rendering the telemetry in the top bar header right beside the 'Secure Connection' lock symbol.
- Fallback Strategy: If browser permissions are denied or unavailable, the system hooks into '`https://ipapi.co/json/`' to resolve geography via client IP address, fetches weather for those IP coordinates, and appends a '(PROXY)' suffix to alert audit grids.



## Ambient Background Audio & YouTube API Integration

To construct an immersive terminal experience, the portal implements a customized ambient background audio system utilizing the YouTube IFrame Player API. Due to strict browser autoplay policies that restrict media playback without active user engagement, the system features a robust User Interaction Guard. On initialization, the terminal dynamically injects the YouTube IFrame API script tag ([https://www.youtube.com/iframe\\_api](https://www.youtube.com/iframe_api)) and instantiates a hidden player linked to a specific Syndicate audio track. The player is configured to loop continuously with keyboard shortcuts and media controls disabled (controls: 0, disablekb: 1). To initiate playback smoothly and comply with browser restrictions, the script binds a one-time activation listener across human interaction triggers (click, keydown, touchstart). Upon the first user interaction, the activation listener un-mutes the player, limits the volume to a comfortable 40% background level, and begins playback immediately. This setup provides a seamless, immersive atmosphere from the moment the operative accesses the terminal.

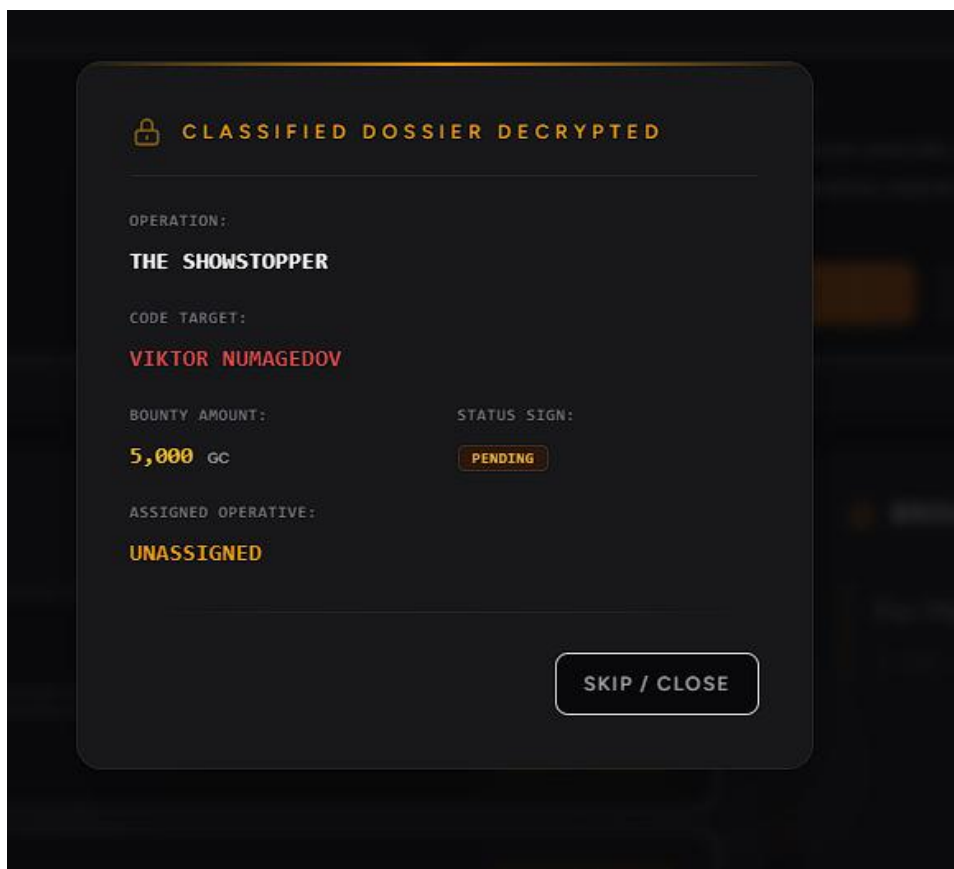
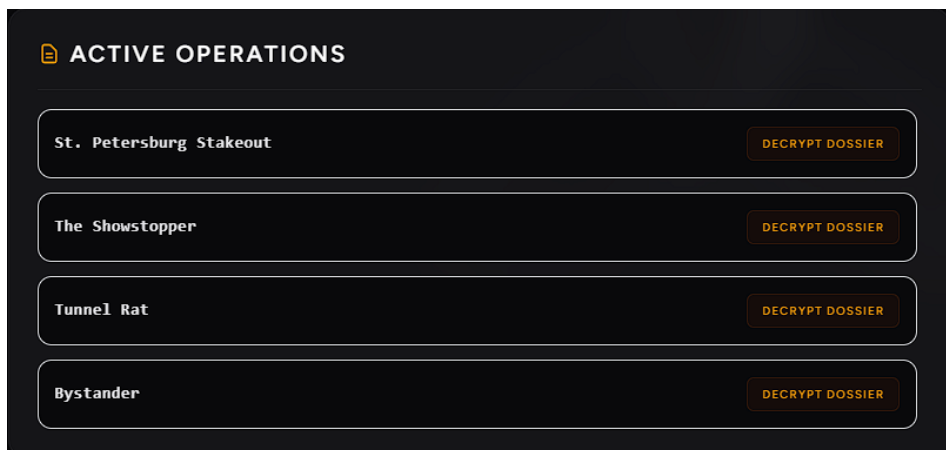
## Unassigned Contract Initialization

Upon a new contract deployment, it is enforced that new operations are published without user assignment ('user\_id = null') and with a default state of 'pending'. Operatives can independently view unassigned contracts and accept them directly on their terminals.

Technical Specifics:

- Admin form is located at route 'admin.contracts.create' and rendered through `admin/contracts/create.blade.php`. The layout has been cleaned to exclude operative assignment selectors, simplifying deployment actions.
- When submitted, `ContractController.php` validates 'title', 'target', and 'bounty' inputs. The controller persists the record to the database initializing status to 'pending' and the user\_id foreign key constraint to null.
- A redirection to the administrator's account management dashboard is triggered with a success notification banner stating: 'NEW CONTRACT INITIALIZED: AWAITING OPERATIVE ACCEPTANCE.'

The screenshot shows a web dashboard with a dark theme. At the top, there is a navigation bar with links for 'DASHBOARD', 'REQUESTS', and 'USERS', and a 'SNAPE' dropdown menu. The main heading is 'INITIALIZE SYNDICATE OPERATIONAL CONTRACT'. Below this, there is a form with three input fields: 'OPERATION TITLE' (with placeholder 'e.g. The Showstopper'), 'TARGET NAME / CODENAME' (with placeholder 'e.g. Viktor Novikov'), and 'BOUNTY REWARD (GOLD COINS)' (with placeholder 'e.g. 15000'). At the bottom of the form, there are two buttons: 'ABORT OPERATION' and 'DEPLOY CONTRACT'.



## Interactive Dossier Decryption Modal

Built a cyberpunk-themed modal interface in the operative's dashboard. Clicking 'Decrypt Dossier' gathers details via data attributes and opens the modal. Button options toggle based on status:

- Pending: Shows 'Accept Contract' and 'Skip'.
- Accepted: Shows 'Mark Completed' and 'Skip'.
- Completed: Shows only 'Skip / Close'.

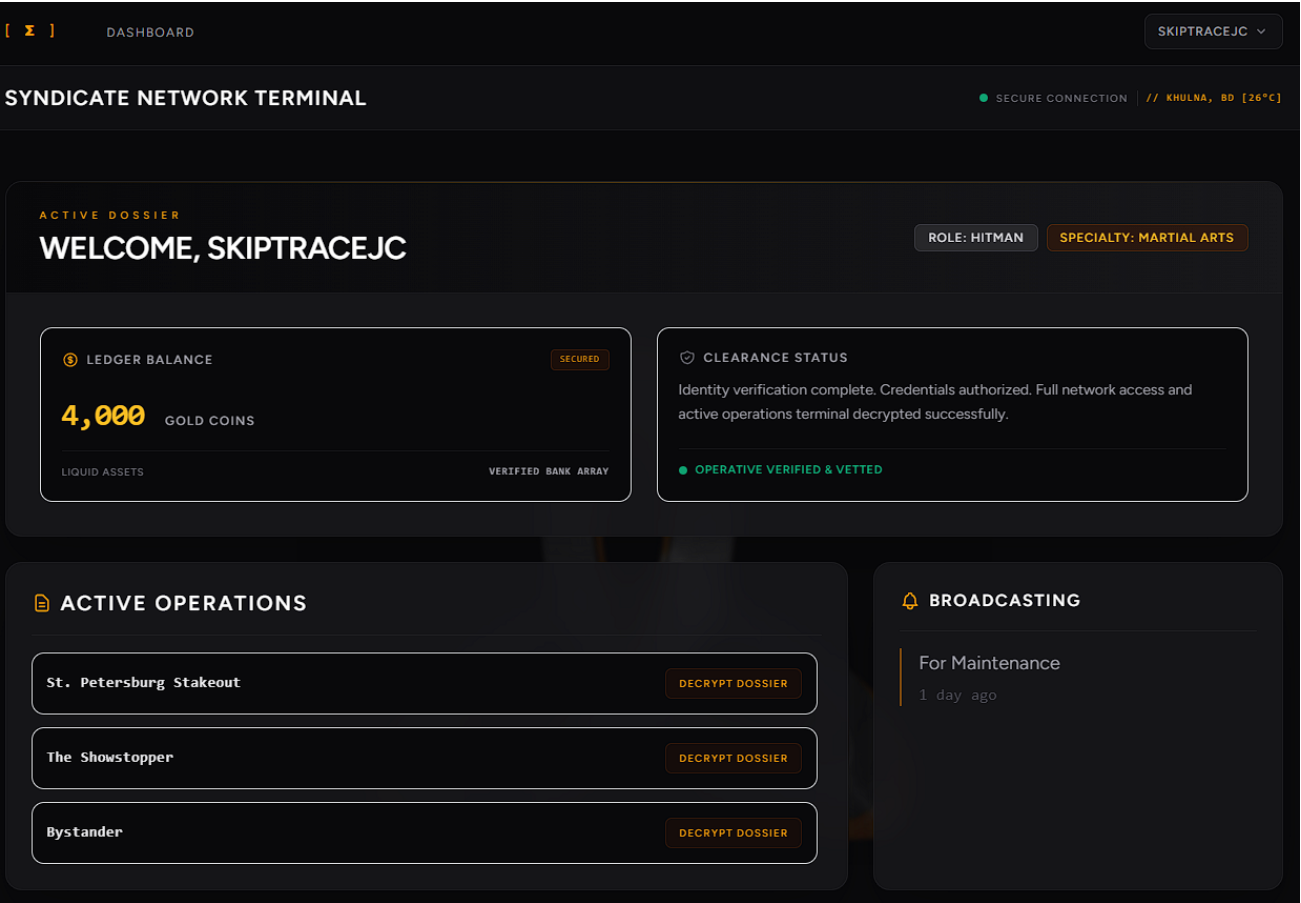
Technical Specifics:

- Modal HTML wrapper is isolated outside the main JavaScript script tag blocks inside `dashboard.blade.php`, avoiding syntax parser failures during client-side rendering.
- Decrypt Dossier triggers are configured as `<button>` elements with datasets storing specific

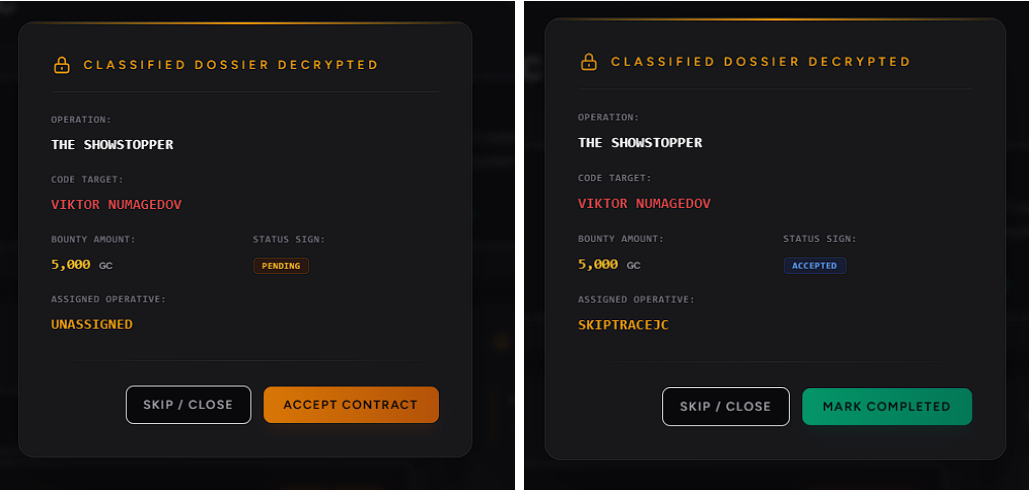
contract fields (data-id, data-title, data-target, data-bounty, data-status, data-assignee).

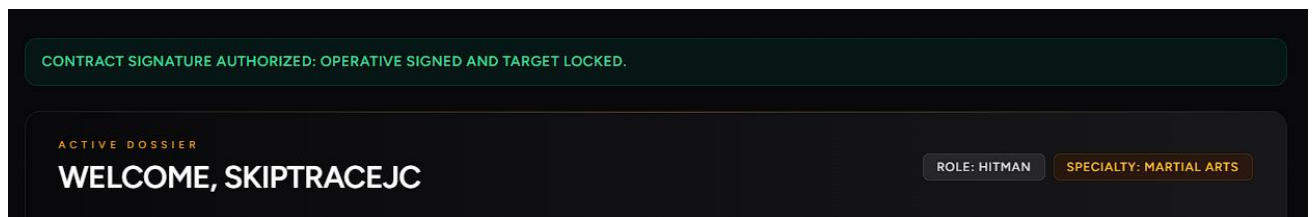
- The event listener parses attributes on click, injects details into the corresponding fields (modal-title, modal-target, modal-bounty, modal-status, modal-assignee), and toggles visual CSS transitions.
- Null safety checks are added inside JavaScript. If action forms (Accept/Complete) are not rendered (for example, on an Admin dashboard), the script executes safely. Form actions are populated dynamically (e.g., pointing accept form to /contracts/{id}/accept).

General hitman dashboard looks like: Here Broadcasting board displays announcements of Syndicate.



Dossier pending and accepted:





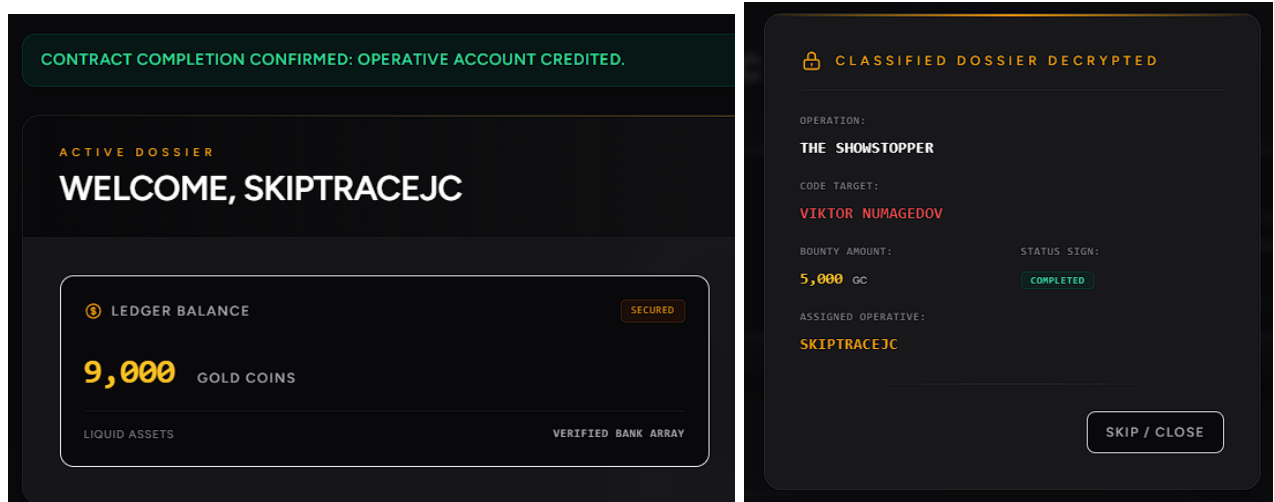
## Bounty Auto-Credit Ledger Logging

Connected the contract completion action to auto-audit. Marking an accepted contract as completed instantly logs the payout transaction in the database ledger with correct timestamps and foreign keys, and redirects back with confirmation alerts.

Technical Specifics:

- The POST request to route 'contracts.complete' verifies ownership of the contract. The ContractController.php complete() method checks if the contract's user\_id matches the active user's ID and status is 'accepted'.
- Upon passing verification, status updates to 'completed'. The controller invokes the Ledger Eloquent model to create a transaction: `Ledger::create(['contract_id' => $contract->id, 'hitman_id' => auth()->id(), 'amount' => $contract->bounty, 'paid_at' => now()])`.
- Ledger mappings are instantly indexed in database records. The operative is redirected back with an audit feedback message: 'CONTRACT COMPLETION CONFIRMED: OPERATIVE ACCOUNT CREDITED.'

Dossier Completed (Balance updated):



## Role-Based Dashboard Contract Visibility

Ensures contract lists are strictly filtered according to authorization role clearances on the main terminal dashboard.

Technical Specifics:

- Admins query and see all contracts (pending, accepted, completed) with eager loaded 'assignee' models. Inside the decrypted dossier, Admins can review who accepted or completed the contract. Action buttons (Accept/Complete) are hidden from Admins.



- Operatives (Clearance 'Hitman') can only see unassigned pending contracts (allowing them to claim) and accepted or completed contracts assigned to them. Contracts owned by other operatives remain strictly hidden.

## Updating Profile Information

User can update information or delete their account.

[ X ]

DASHBOARD

OPERATIVE PROFILE DOSSIER

IDENTITY SIGNATURE MATRIX

Modify your core network clearance parameters, active codename assignment, and routing email coordinates.

ASSIGNED CODENAME

skiptraceJC

SECURE COMMUNICATION CHANNEL

jackiechan@yahoo.com

COMMIT MATRIX CHANGES

UPDATE ACCESS CIPHER

Ensure your account is using a complex, randomized key configuration to guarantee network security containment.

CURRENT CIPHER

NEW CIPHER GENERATION

CONFIRM CIPHER SIGNATURE

COMMIT CHANGE

EMERGENCY PROFILE LIQUIDATION

CRITICAL WARNING: Activating this routine permanently purges all network access signatures, database logs, and operational history from the Syndicate mainframe. This process is irreversible.

PURGE DOSSIER ACCOUNT

## User Management from Admin

Admin can check all the users and terminate them permanently at any time. Also admin can create a new user account for hitman which will automatically be accepted/verified.

[ X ]

DASHBOARD

REQUESTS

USERS

USERS

SNAP E

USER MANAGEMENT REGISTRY

INITIALIZE CONTRACT

CREATE ACCOUNT

| CODENAME    | EMAIL                          | ROLE   | SPECIALTY    | ACTIONS   |
|-------------|--------------------------------|--------|--------------|-----------|
| GHOST       | ghost@taskforce141.com         | HITMAN | Infiltration | TERMINATE |
| SKIPTRACEJC | jackiechan@yahoo.com           | HITMAN | Martial Arts | TERMINATE |
| SNAP E      | salehin2207044@stud.kuet.ac.bd | ADMIN  | Shadow       | TERMINATE |

## Pending Request Control

Admin can observe all the pending, accepted, rejected registration requests. If a request is declined, that user can never log in or request again which is ensured by database operations.

[ Σ ]

DASHBOARDREQUESTSUSERS

SNAPE

PENDING OPERATIVE REQUESTS

APPLICATION QUEUE

| CODENAME                             | REFERRAL | STATUS   | ACTIONS           |
|--------------------------------------|----------|----------|-------------------|
| <b>Xander Cage</b><br>Extreme Riding | Ghost    | PENDING  | APPROVE<br>REJECT |
| <b>Ghost</b><br>Infiltration         | Snake    | APPROVED | VERIFIED BY SNAPE |
| <b>Skiptrace3C</b><br>Martial Arts   | Agent 47 | APPROVED | VERIFIED BY SNAPE |
| <b>John Wick</b><br>Assault          | Agent 47 | REJECTED | VERIFIED BY SNAPE |

## Conclusion

The implementation of the unassigned contract creation workflow and operative claiming mechanisms has greatly enhanced system usability and decentralization, adhering strictly to the Syndicate's operational guidelines. By isolating dossier details behind secure modal panels and tracking payouts automatically inside the ledger audit tables, the system guarantees high operational secrecy and absolute financial transparency.

The project displays clear usages of Laravel Breeze secure authentication to keep accesses secured, Middleware usages to protect different pages from unauthorized access, CSS for design, JavaScript for validate/ensuring form fields, API for background music and location, region and temperature(weather) tracking. Thus it is a complete package of web programming associated stuffs.